

Torqline

Service appointments and repair orders for car and bike dealerships

Complete system design, documentation and interview guide

Java 21 · Spring Boot 4 · Kafka · PostgreSQL · Redis · React · github.com/chethansv23/torqline

Contents

1. Interview guide
 2. Project overview (README)
 3. High-level design
-

4. Low-level design
5. ADR-001: Double-booking prevention
6. ADR-002: Transactional outbox
7. ADR-003: Database per service
8. API reference
9. Testing guide
10. User guide
11. AI review log
12. Article

Interview guide

Likely questions about Torqline, with short answers you can give in your own words.

The 60-second pitch

"Torqline is a service-booking and repair-order platform for dealerships that service both cars and bikes. Customers book a slot, the workshop checks the vehicle in, a job card opens automatically, parts are reserved from stock, and the customer gets SMS updates and a GST invoice. It's five Spring Boot microservices with PostgreSQL, Kafka and Redis, plus a React UI, all running with one command. The interesting part is correctness under concurrency: I guaranteed no double booking with a database constraint, found a deadlock with a load test, fixed it, and proved the fix with tests."

Architecture

Why microservices and not a monolith? Booking, repair orders, inventory and notifications change for different reasons and scale differently; availability reads are far hotter than invoicing. Separate services with their own databases can be deployed and scaled independently. The trade-off is eventual consistency, which I handle with the outbox, idempotent consumers and a saga. For a small team I'd honestly start as a modular monolith with the same boundaries.

How do services communicate? Asynchronously over Kafka. For example, the appointment service publishes `AppointmentCheckedIn` and the repair-order service reacts by opening a job card. No service calls another service's database. The gateway is the only synchronous entry point.

Why database per service? Independent schema changes and deployments, and no hidden coupling through shared tables. Data another service needs travels in events; the repair order keeps its own copy of customer and vehicle details.

Double booking (the core question)

How do you prevent two customers getting the same bay? A PostgreSQL exclusion constraint: `EXCLUDE USING gist (bay_id WITH =, tstzrange(slot_start, slot_end) WITH &&) WHERE status <> 'CANCELLED'`. The database rejects any overlapping booking on the same bay, regardless of which instance or code path writes it. The service tries likely-free bays in turn and returns 409 when none are left.

Why not a Redis lock or a check in Java? A check-then-insert in Java has a race window. A Redis lock can be lost on failover or TTL expiry, and it still needs a DB check. Putting the rule in the database that owns the data is the simplest thing that's always correct.

What went wrong? Under a 50-user k6 race, concurrent inserts deadlocked while checking the constraint: each transaction waited on the other's uncommitted row. I got 1 booking and 49 errors, and connection-pool starvation pushed read latency to 23 s. I first made the integration test fail the same way (50 threads, bigger pool), then added `pg_advisory_xact_lock(bay_id)` before the insert. Inserts for one bay now queue, and each loser fails fast with a clean violation. Result: exactly 4 bookings for 4 bays, 46 clean 409s, p95 of 2.6 ms.

Why can't the advisory lock itself deadlock? Each transaction takes at most one such lock, and it's released at commit or rollback, so no cycle can form.

Reliability

What's the dual-write problem and how does the outbox solve it? Writing to the DB and to Kafka separately can leave one done and the other not, which means lost or phantom events. I write the event to an `outbox_event` table in the same

transaction as the state change. A relay polls with `FOR UPDATE SKIP LOCKED` , publishes, waits for acks, then marks the rows published.

Exactly-once? Delivery is at-least-once. Effects are exactly-once because each consumer inserts the event ID into `processed_event` in the same transaction as its work and skips duplicates. Kafka transactions wouldn't help because they don't cover the Postgres write.

What if a message keeps failing? Three retries one second apart, then it goes to a dead-letter topic (`<topic>-dlt`) for inspection.

Idempotent booking API? Clients send an `Idempotency-Key` header. A repeat returns the original booking with 200 instead of 201, and a unique constraint on the key handles two identical requests racing. Known gap: I don't yet reject a reused key with a different body.

The parts saga

How does it work? The repair order moves to `PARTS_PENDING` and publishes `PartsReservationRequested` . Inventory locks the part rows (`SELECT ... FOR UPDATE` , **ordered by SKU** to avoid deadlocks) and reserves everything or nothing. It replies `PartsReserved` or `PartsReservationFailed` with a reason, and the order goes back to `IN_PROGRESS`. On completion, stock is consumed. On cancellation, a compensating action releases it.

Orchestration or choreography? Choreography. There are only two participants and the repair order already owns the state. I'd add an orchestrator if payments or warranty joined the flow.

How is ordering guaranteed? Repair-order events are keyed by repair-order ID, so all events for one order land on one partition in order.

What about a late reply after a cancel? The aggregate ignores replies that don't match a pending request, and a unit test covers it.

Performance and caching

How is availability fast? Redis caches each branch's booked intervals per day for 60 s; availability is computed in memory from those. One key per branch-day means one delete after any change. If Redis is down it falls back to Postgres. It measures p95 of about 3 ms at 200 req/s.

Isn't stale cache dangerous? Only for display. The database decides every booking, so the worst case is a 409 and a refreshed grid.

How would you scale it? All services are stateless, so add instances. The outbox relay supports several instances through `SKIP LOCKED` . Beyond that: more Kafka partitions, read replicas for availability, and bays partitioned by dealer.

Data and domain

How are cars and bikes modelled? Bays have a vehicle type; `ServiceType` has a duration per vehicle type (null means not offered); `VehicleType` carries the labour rate; parts have a fitment of car, bike or universal.

Timezones? Stored as `timestamp tz` ; the API takes the dealer's local time and each dealer has a timezone.

Migrations? Flyway with `ddl-auto=validate` , so the schema is reviewed SQL and Hibernate only checks it.

Testing

How did you test it? 59 backend tests: unit tests (Mockito), MockMvc for the HTTP contract, and Testcontainers race tests on real Postgres. 29 frontend tests with Vitest and React Testing Library, an end-to-end script, and k6 with pass/fail

thresholds.

A bug your tests found? Three. The deadlock. A k6 script that could never fail, because each virtual user picked its own random date, so nothing actually raced. And validation errors that returned no field details, because Spring 7 throws a different exception when a controller also validates a header.

Using AI (this posting asks about it)

How did you use AI? To draft the HLD and LLD, generate code, and review designs. I logged every significant suggestion in `AI-REVIEW.md` with how I checked it. I rejected a Redis lock, direct Kafka publishing and `ddl-auto=update`. The deadlock and the broken load test are examples of AI output that looked right and failed under measurement.

How do you judge whether AI output is right? Ask "how would this fail?", write a test that would catch it, watch it fail, then fix it. And sanity-check numbers: 50 of 50 booked with 4 bays is impossible.

What would you do next?

Kubernetes and AWS (EKS, RDS, MSK, ElastiCache), Grafana dashboards for conflicts and outbox lag, authentication with roles, rate limiting at the gateway, and no-show handling.

Question bank

Short answers. Say them in your own words and link each back to Torqline where you can.

A. High-level design follow-ups

1. Walk me through the request path when a customer books. Browser → nginx (web) → gateway → appointment-service. It validates the request, reads cached busy slots from Redis, then for each candidate bay opens a short transaction: advisory lock, insert appointment and outbox row, commit. It evicts the cache and returns 201. The relay then publishes `AppointmentBooked`, and notification-service sends the SMS.

2. What are the functional and non-functional requirements? Functional: availability, booking, cancel, check-in, job cards, parts reservation, invoicing, notifications. Non-functional: zero double bookings, availability p95 under 150 ms, no lost events, tolerance of Redis or Kafka outages, horizontal scaling, observability.

3. Estimate the load. Example: 500 dealers × 10 bays × 16 slots/day is about 80k bookings/day, around 1 write/s on average and perhaps 20/s at peak. Availability reads might be 50× that, about 1,000/s at peak. One Postgres instance handles the writes, and Redis absorbs the reads.

4. Where are the single points of failure and how would you remove them? Locally: one Postgres, one Kafka broker, one Redis. In production: RDS Multi-AZ, MSK with 3 brokers and replication factor 3, ElastiCache with a replica, and 2+ instances of every service behind a load balancer.

5. What happens if Kafka is down? Bookings still succeed; events wait in the outbox table and ship when Kafka returns. Notifications and job cards are delayed, not lost.

6. What happens if Redis is down? Availability reads go straight to Postgres; it's slower but correct. Bookings are unaffected.

7. What happens if Postgres is down? That service can't serve writes and fails health checks, so no traffic is routed to it. It's the source of truth, so that's the right failure mode rather than accepting bookings you can't guarantee.

- 8. What if one service is slow?** Others aren't blocked because they talk asynchronously. At the gateway I'd add timeouts, a circuit breaker (Resilience4j) and rate limiting.
- 9. Why is the gateway useful?** One entry point, routing, request IDs, and a place for auth, rate limiting and CORS later.
- 10. How would you add payments?** A payment-service that consumes `RepairOrderCompleted`, creates a payment intent and publishes `PaymentCaptured`, making it another step in the saga. At that point I'd consider an orchestrator.
- 11. How would you add multi-region?** Partition by dealer: each dealer's data lives in one home region, so bookings never need cross-region locks. Replicate read-only data such as the catalogue.
- 12. Sync vs async: when do you use which?** Sync when the user needs the answer now (booking, availability). Async when work can happen afterwards and should survive failures (job card, parts, SMS).
- 13. Consistency model?** Strong inside each service's database; eventual across services. The user sees `PARTS_PENDING` while the saga completes.

B. Low-level design follow-ups

- 14. Explain the booking schema.** `appointment` holds bay, slot start and end as `timestampz`, status, an idempotency key (unique) and a version. The exclusion constraint on `(bay_id, tstzrange)` excludes cancelled rows, so cancelling frees the bay immediately.
- 15. Why `[]` ranges?** Half-open ranges let a 10:00-11:00 booking sit next to an 11:00-12:00 one without conflicting.
- 16. Why a GiST index?** Range overlap (`&&`) needs GiST. `btree_gist` lets it combine equality on `bay_id` with range overlap in one index.
- 17. How does the service choose a bay?** Bays that look free in the cached busy list come first; it inserts on each in turn and moves on when a bay turns out to be taken. This keeps the number of failed attempts low.
- 18. Why a separate transaction per bay attempt?** After a constraint violation Postgres aborts the transaction, so each attempt needs a fresh one.
- 19. Walk me through the repair-order state machine.** `OPEN` → `IN_PROGRESS` → `PARTS_PENDING` ↔ `IN_PROGRESS` → `COMPLETED`; `CANCELLED` from any non-terminal state. Allowed moves live in `RepairOrderStatus.next()`, and the aggregate refuses anything else with 409.
- 20. How are invoices calculated?** Labour = duration × rate (car ₹800/h, bike ₹400/h). Add reserved parts only, then 18% GST. It uses `BigDecimal` with `HALF_UP` rounding, because floating point is wrong for money.
- 21. Optimistic vs pessimistic locking: where and why?** Optimistic (`@Version`) on aggregates edited by people, where conflicts are rare and should be a 409. Pessimistic (`FOR UPDATE`) on stock rows, where contention is expected and every request must see the true quantity.
- 22. How do you avoid deadlocks in inventory?** Always lock parts in SKU order, so two transactions never hold locks in opposite orders.
- 23. What's in an event?** JSON payload keyed by aggregate ID, with `eventId` and `eventType` headers. Plain JSON keeps services independent of each other's classes.
- 24. How is schema evolution handled for events?** Only additive changes (new optional fields); consumers ignore unknown fields. A breaking change gets a new event type or version header. A schema registry (Avro/Protobuf) would enforce this at scale.
- 25. How do you clean up the outbox?** Published rows older than 7 days are deleted hourly.

26. What indexes did you add and why? `(dealer_id, slot_start)` for a day's appointments, a partial index on unpublished outbox rows, `(dealer_id, status, opened_at)` for the job board, and `(repair_order_id, status)` for reservations.

27. Why Flyway with `validate`? Schema changes are reviewed SQL. Hibernate only checks that the entities match, so the app refuses to start with a wrong schema instead of silently altering it.

28. How does the error model work? RFC 9457 ProblemDetail with a stable `code`, so clients branch on the code, not on the message.

C. Java and Spring Boot

29. Why records? Immutable DTOs and events with equals/hashCode for free and less boilerplate.

30. `@Transactional` vs `TransactionTemplate`: why both? `@Transactional` for simple service methods. `TransactionTemplate` where I need several separate transactions in one method, like one per bay attempt, or in shared library code.

31. Pitfalls of `@Transactional`? Self-invocation bypasses the proxy; it only works on public methods; checked exceptions don't roll back by default; long transactions hold locks.

32. What is `open-in-view` and why disable it? It keeps the Hibernate session open during view rendering, which hides lazy-loading problems and holds connections longer. I turned it off.

33. N+1 queries: how would you spot and fix them? Enable SQL logging or Hibernate statistics. Fix with fetch joins, entity graphs, batch fetching or DTO projections. I used a JPQL constructor projection for busy intervals.

34. How does the shared library get wired in? Spring Boot auto-configuration: `torqline-common` registers its beans through `AutoConfiguration.imports`, so every service gets the outbox, idempotency guard and error handler just by adding the dependency.

35. How does validation work? Bean Validation annotations on request records, `@Valid` on the controller, and one handler that turns both validation exception types into the same 400 response.

36. Virtual threads? Java 21 supports them (`spring.threads.virtual.enabled=true`). They help I/O-bound request handling, but DB connections are still the limit, so the pool size matters more.

37. How is config managed? `application.yml` defaults for local dev, overridden by environment variables in Docker. In the cloud: secrets from Secrets Manager, config via environment or Spring Cloud Config.

D. Kafka

38. Why Kafka over RabbitMQ? Durable, replayable log with per-key ordering, and consumers can re-read history. RabbitMQ would also work for this scale; Kafka fits event-driven microservices and replay.

39. Partitions and ordering? Ordering is guaranteed only within a partition. Keying by aggregate ID keeps one order's events in sequence. Three partitions allow up to three consumers per group.

40. Consumer groups? Each service has its own group, so every service gets every event; instances of one service share the partitions.

41. What is a rebalance and why does it cause duplicates? When consumers join or leave, partitions move. Offsets that weren't committed are re-delivered, which is why the consumers are idempotent.

42. `acks=all` and idempotent producer? `acks=all` waits for all in-sync replicas; the idempotent producer prevents duplicates caused by producer retries.

43. Dead-letter topic: what do you do with it? Alert on it, inspect the message, fix the bug, and replay it to the main topic.

44. Consumer lag: how do you monitor it? Kafka UI locally; in production, the consumer lag metric in Prometheus with alerts.

E. PostgreSQL

45. Isolation level used? READ COMMITTED (the default). The exclusion constraint and row locks give the guarantees; SERIALIZABLE would add retries without adding safety here.

46. What are advisory locks? Application-defined locks on a number, managed by Postgres. `pg_advisory_xact_lock` is released automatically at the end of the transaction.

47. SKIP LOCKED ? Skips rows locked by others instead of waiting, so parallel workers each take different outbox rows.

48. How would you scale Postgres? Read replicas for reads, connection pooling (PgBouncer), partitioning appointments by date or dealer, and archiving old rows.

49. How did you size the connection pool? Default Hikari pool of 10. The deadlock showed that waiting transactions hold connections, so short transactions matter more than a bigger pool.

F. Redis and caching

50. Cache-aside vs write-through? Cache-aside (read-through): load on a miss, evict on a write. It's simple and the DB stays the source of truth.

51. Cache stampede? Many requests missing at once. With a 60 s TTL per dealer-day it's small; fixes are a short lock on load or early refresh with jitter.

52. Why not cache the final availability? There are too many combinations (vehicle × service). Raw intervals give one key and one evict.

G. Docker, Kubernetes and AWS

53. Explain the Dockerfile. A multi-stage build: Maven builds the jar with a cached dependency directory, and a slim JRE image runs it as a non-root user with memory-aware JVM flags.

54. How would you deploy to Kubernetes? A Deployment and Service per microservice, readiness and liveness probes on Actuator, a HorizontalPodAutoscaler, ConfigMaps and Secrets, Ingress for the gateway and web, and managed Postgres, Kafka and Redis.

55. Readiness vs liveness? Readiness: can I take traffic now (DB reachable)? Liveness: am I stuck and should I be restarted? Mixing them up causes restart loops.

56. Zero-downtime deploys? Rolling updates with readiness probes, backward-compatible DB migrations (expand, then contract), and additive event changes.

57. Observability? Metrics (Micrometer → Prometheus → Grafana), logs with a request ID, and tracing (OpenTelemetry). Key alerts: 5xx rate, booking 409 rate, outbox lag, consumer lag, DLT count.

H. Security

58. How would you add authentication? OAuth2/OIDC (e.g. Keycloak or Cognito). The gateway validates JWTs and services check roles (customer vs advisor).

59. What other security gaps exist today? No auth, no rate limiting, default DB passwords, and no PII masking in logs. Fixes: auth, a gateway rate limiter, secrets from a vault, and masking phone numbers.

60. SQL injection? Prevented by JPA and parameterised JdbcTemplate queries; no string-built SQL.

I. Frontend

61. Why React without Redux? The state is mostly server data; each screen polls the API. A library like TanStack Query would be the next step.

62. Why polling and not WebSockets? Simple, and fine for a workshop board. At scale I'd push updates with Server-Sent Events or WebSockets fed from Kafka.

63. How does the UI stay in sync after a rebuild? `index.html` is served with no-cache and the hashed assets are immutable.

64. How is the frontend tested? Vitest and React Testing Library, with the API module mocked. Tests query by role and text, the way users see the page.

J. Testing

65. Unit vs integration: where's the line? Unit tests for pure rules (slot grid, pricing, state machine). Integration tests with Testcontainers where the database is the logic (the exclusion constraint).

66. Why Testcontainers over H2? H2 doesn't support exclusion constraints, GiST or advisory locks, so the key guarantee could only be tested on real Postgres.

67. How do you test concurrency? Start N threads behind a latch so they fire together, then assert the exact number of successes and that every failure is the expected 409.

68. What would you add? Contract tests for events, an end-to-end test of the saga with Kafka in Testcontainers, and mutation testing.

K. Scenario questions

69. A customer says they were double-booked. How do you investigate? Check the DB: the constraint makes it impossible for one bay, so look for two bookings on different bays or a cancelled-and-rebooked slot. Use the request ID to trace the logs, and check whether the UI showed a stale cache.

70. Inventory shows negative stock. It can't: a CHECK constraint enforces $0 \leq \text{reserved} \leq \text{on_hand}$. Check for manual SQL edits or a failed migration.

71. SMS messages are arriving twice. Check the `processed_event` records and the consumer logs for skipped duplicates. If notifications were saved twice, the idempotency insert isn't in the same transaction as the handler.

72. Bookings suddenly slow at 9 AM. Look at pool usage, lock waits (`pg_stat_activity`), the Redis hit rate and GC. Likely causes are cache misses at the start of the day or lock contention on popular bays.

73. Outbox table is growing. The relay can't publish: Kafka is down, or a poison row keeps failing the batch. Alert on unpublished row age, and send failing rows one by one or park them.

74. The product team wants 15-minute slots. Change the grid constant and the durations; the constraint works for any interval. Migrate existing data if needed.

75. A dealer adds a bay. Insert a row into `service_bay`. It appears in availability once the cache for that day expires or is evicted.

L. Behavioural

76. Hardest problem in this project? The deadlock. I didn't guess at a fix; I reproduced it in a test first, read the Postgres log, fixed the cause, and measured before and after.

77. A trade-off you made? Eventual consistency for job cards and SMS in exchange for independent services, with outbox and idempotency so nothing is lost.

78. What would you do differently? Start as a modular monolith, add auth from day one, and add contract tests for events earlier.

79. How do you mentor juniors using a project like this? ADRs to explain why, tests that document behaviour, and the AI review log to show how to question generated code.

80. How do you review code? Correctness first (races, transactions, error paths), then tests, then readability. I ask "how does this fail?" for every external call.

Torqline

Service appointments and repair orders for car and bike dealerships

Java 21 · Spring Boot 4 · Kafka · PostgreSQL · Redis · React 19 · TypeScript · Docker

 **Torqline**

Book service
Customer

Workshop
Service advisor

Inventory
Parts desk

Torqline Indiranagar ▾

1 Your vehicle

**Bike / scooter**
2 stands at this branch

**Car**
3 lifts at this branch

2 Service

General service
Periodic maintenance service
60 MIN

Oil change
Engine oil and filter change
30 MIN

Brake service
Brake inspection and pad replacement
60 MIN

Tyre replacement
Tyre replacement and balancing
30 MIN

Clutch overhaul
Clutch overhaul
90 MIN

Chain sprocket
Chain and sprocket kit replacement
60 MIN

3 Date and time

Today
26 Sept

**Tomorrow
27 Sept**

Mon
28 Sept

Tue
29 Sept

Wed
30 Sept

Thu
1 Oct

Fri
2 Oct

Sat
3 Oct

9:00 AM
1 of 2 free

9:30 AM
1 of 2 free

10:00 AM
1 of 2 free

**10:30 AM
1 of 2 free**

11:00 AM
2 of 2 free

11:30 AM
2 of 2 free

12:00 PM
2 of 2 free

12:30 PM
2 of 2 free

1:00 PM
2 of 2 free

1:30 PM
2 of 2 free

2:00 PM
2 of 2 free

2:30 PM
2 of 2 free

3:00 PM
2 of 2 free

3:30 PM
2 of 2 free

4:00 PM
2 of 2 free

4:30 PM
2 of 2 free

5:00 PM
2 of 2 free

4 Your details

Name
Chethan S

Mobile
98450 22222

Registration number
KA 51 EZ 7788

Make
TVS

Model
Apache RTR 160

Notes
Front brake squeals

General service
 27 Sept, 10:30 AM to 11:30 AM

Confirm booking

Torqline is a small but realistic dealership service platform. A customer books a slot for their **car or bike**, the workshop checks the vehicle in, a job card opens automatically, parts are reserved from the dealer's stock, and the customer gets an itemised GST invoice and SMS updates along the way.

It's built as event-driven microservices and runs on a laptop with one command. It focuses on the problems real workshops have, such as double-booked bays, retries that create duplicate bookings, and jobs promising parts that aren't on the shelf, and it solves each one in a way that's tested and measured.

Contents

- [Features](#)
- [Screenshots](#)
- [Quick start](#)
- [Architecture](#)
- [Engineering highlights](#)

Torqline · 11 / 47

- [Testing](#)
- [Project structure](#)
- [Documentation](#)
- [Troubleshooting](#)
- [Roadmap](#)

Features

For	What they can do
Customers	Choose bike or car, see only the services that apply to that vehicle, pick a day and a free slot, book, and get an SMS confirmation
Service advisors	See the day's appointments, check vehicles in (with odometer reading), run a live job-card board, assign technicians, request parts, complete jobs and print invoices
Parts desk	See stock per part with reserved vs available, get low-stock alerts, restock
Everyone	A live feed of every SMS and email the system sent

How cars and bikes differ

	Car	Bike
Bays	Car lifts	Bike stands
General service	120 min	60 min
Labour rate	₹800 / hour	₹400 / hour
Vehicle-only jobs	Wheel alignment, AC service	Chain and sprocket kit
Parts	Car parts + universal consumables	Bike parts + universal consumables

Screenshots

Workshop board

Torqline

Book service

Workshop

Inventory

Torqline Indranagar

Appointments

Check a vehicle in when it arrives. That opens a job card on the board below.

Today

Tomorrow

Mon

Tue

Wed

Thu

Fri

Sat

TIME	VEHICLE	CUSTOMER	SERVICE	STATUS
9:00 AM	KA83HB1234 Royal Enfield Classic 350	Asha Rao 9845020208	General service	Checked in
9:00 AM	KA81M4321 Maruti Suzuki Baleno	Asha Rao 9845020208	AC service	Checked in
10:00 AM	KA85TR4242 Bajaj Pulsar NS200	Test Rider 9845011111	General service	Checked in

Workshop board

Updates live. Parts requests are handled asynchronously by the inventory service over Kafka.

Checked in

In progress

Waiting for parts

Ready for pickup

Waiting for a technician

On the lift or stand

Inventory is reserving stock

Invoice

Parts inventory

Torqline

Book service

Workshop

Inventory

Torqline Indranagar

Parts inventory

Reserved parts are promised to open job cards. They leave the shelf when the job completes, or return to stock if it's cancelled.

AB

Car

Bike

Universal

2 part(s) at or below reorder level. The service manager has been emailed.

PART	FITS	PRICE	STOCK	AVAILABLE
AC refrigerant R134a, 500 g	CAR	₹1,500.00	20 on hand	20 +10
Air filter (bike)	BIKE	₹320.00	8 on hand	8 +10
Engine air filter (car)	CAR	₹650.00	20 on hand	20 +10
Brake fluid DOT 4, 500 ml	UNIVERSAL	₹420.00	14 on hand	14 +10
Disc brake pad set (bike)	BIKE	₹650.00	7 on hand	7 +10
Front brake pad set (car)	CAR	₹2,400.00	20 on hand	20 +10
Cabin AC filter	CAR	₹550.00	20 on hand	20 +10
Chain and sprocket kit	BIKE	₹2,200.00	11 on hand	11 +10
Clutch plate set (bike)	BIKE	₹1,400.00	8 on hand	8 +10

Printable GST invoice

Back to workshop

Print / Save as PDF

TAX INVOICE

RO-2026-001002 - 26 Sept 2026, 9:37 pm

Torqline Indranagar

Test Rider

Bengaluru

9845011111

Blue KA85TR4242 - Bajaj Pulsar NS200 - Technician Ravi K

ITEM	QTY	RATE	AMOUNT
Labour: General service	1	₹400.00	₹400.00
Disc brake pad set (Bike)	1	₹650.00	₹650.00
Engine oil 10W-30, 1L	1	₹450.00	₹450.00
Subtotal			₹1,500.00
GST 18%			₹270.00
Total			₹1,770.00

Thank you for servicing with Torqline. This is a computer-generated invoice.

Messages sent (SMS / email)

Torqline

Book service

Workshop

Inventory

Torqline Indranagar

Parts inventory

Reserved parts are promised to open job cards. They leave the shelf when the job completes, or return to stock if it's cancelled.

AB

Car

Bike

Universal

2 part(s) at or below reorder level. The service manager has been emailed.

PART	FITS	PRICE	STOCK	AVAILABLE
AC refrigerant R134a, 500 g	CAR	₹1,500.00	20 on hand	20 +10
Air filter (bike)	BIKE	₹320.00	8 on hand	8 +10
Engine air filter (car)	CAR	₹650.00	20 on hand	20 +10
Brake fluid DOT 4, 500 ml	UNIVERSAL	₹420.00	14 on hand	14 +10
Disc brake pad set (bike)	BIKE	₹650.00	7 on hand	7 +10
Front brake pad set (car)	CAR	₹2,400.00	20 on hand	20 +10
Cabin AC filter	CAR	₹550.00	20 on hand	20 +10
Chain and sprocket kit	BIKE	₹2,200.00	11 on hand	11 +10
Clutch plate set (bike)	BIKE	₹1,400.00	8 on hand	8 +10

Messages sent

SMS and email from notification service

SAVE

9845020208

15m ago

Hi Asha Rao, we have received KA07M4321. We'll message you when the job card is open.

Appointment created

SAVE

9845020208

15m ago

Hi Asha Rao, we have received KA07M4321. We'll message you when the job card is open.

Appointment checked in

SAVE

9845011111

20m ago

Low stock at TO-BLR-IND: Engine oil 10W-30, 1L (DL-10W30-1L) has 6 available, reorder level 8.

Part low stock

SAVE

9845011111

20m ago

Good news Test Rider! KA05TR4242 is ready for pickup. Invoice RO-2026-001002 total Rs 1770.00 (incl. GST).

Repair order completed

SAVE

9845011111

20m ago

Job card RO-2026-001002 opened for KA05TR4242. Track progress with this number.

Repair order created

SAVE

9845011111

20m ago

Hi Test Rider, we have received KA05TR4242. We'll message you when the job card is open.

Quick start

You need: [Docker Desktop](#) with at least 6 GB of memory. Java, Maven and Node are **not** required; everything builds inside Docker.

make up

The first build takes 1-2 minutes. Then open <http://localhost:3000>.

URL	What
http://localhost:3000	Web app
http://localhost:8080/api	REST API through the gateway
http://localhost:8081/swagger-ui.html	API docs for appointments (8082 repair orders, 8083 inventory, 8084 notifications)
http://localhost:8090	Kafka UI (run make too1s first)
<code>localhost:5433</code>	PostgreSQL, user and password <code>torqline</code>

A two-minute tour:

- 1. **Book service:** pick *Bike*, *General service*, tomorrow, any slot, then confirm. The 🛎 lights up with the SMS.
- 2. **Workshop:** click **Check in** on that appointment. A job card appears on the board.
- 3. **Start work** → **Add parts** (engine oil + brake pads) → the card waits briefly while inventory reserves the stock.
- 4. **Complete** → **Invoice** → **Print**. You get an itemised invoice with 18% GST.
- 5. **Inventory:** the stock you used is gone, and a low-stock alert may have been emailed to the manager.

Things to try: book the same bike slot three times at Indiranagar (it has only two bike stands), or request two chain kits at Indiranagar (only one is in stock). See the [user guide](#) for more.

Everyday commands

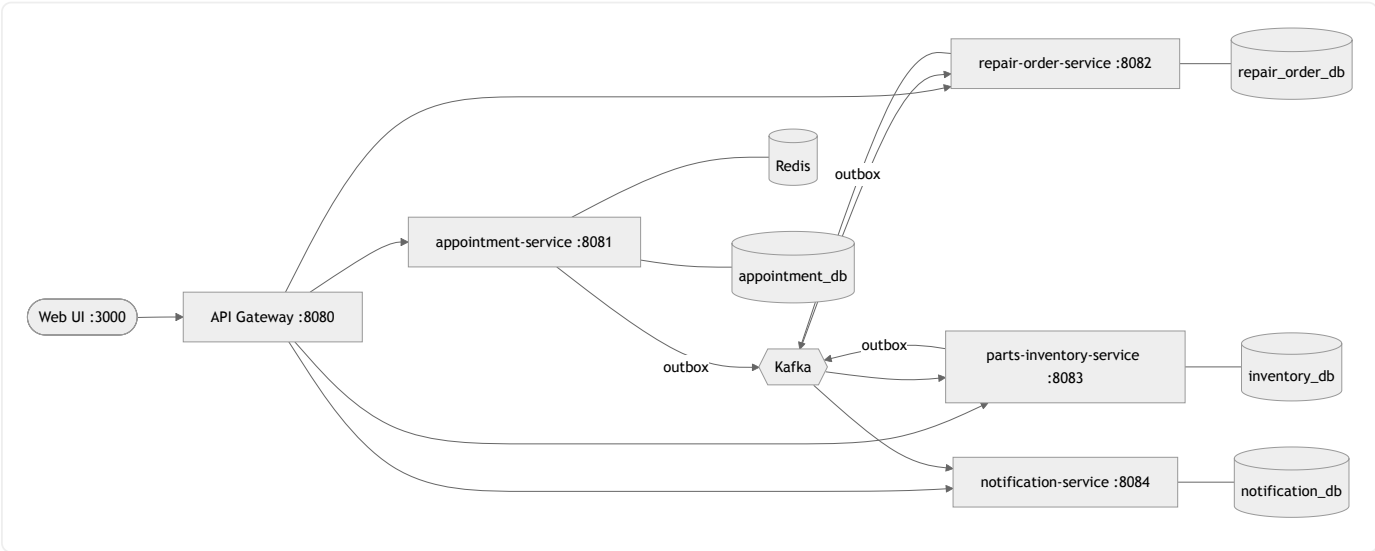
```
make help      # list all commands
make demo      # the same end-to-end flow as a script against the API
make test      # all backend and frontend tests
make logs      # follow service logs
make down      # stop (keeps data)
make reset     # stop and delete all data
```

Developing in an IDE

```
make infra      # start only Postgres, Redis and Kafka
```

Run any `*Application` class from IntelliJ (JDK 21, e.g. `brew install --cask temurin@21`), and run the UI with hot reload using `make ui-dev` (<http://localhost:5173>). Defaults in each `application.yml` point at `localhost`.

Architecture



Service	Responsibility
gateway	Single entry point, routing, request IDs
appointment-service	Dealers, bays, slot availability, booking, check-in, cancellation
repair-order-service	Job cards, status workflow, parts requests, invoice totals
parts-inventory-service	Stock, reservations, consumption, low-stock alerts
notification-service	Turns events into customer SMS and manager emails
web	React UI served by nginx, which also proxies <code>/api</code>

Each service owns its own database. Services talk only through events, never through each other's tables. The full design is in the [high-level design](#) and the [low-level design](#).

Engineering highlights

- 1. No double booking, proven under load.** A PostgreSQL exclusion constraint makes it impossible for one bay to hold two overlapping bookings, even across several service instances. Load testing found that this alone **deadlocks** under heavy contention: 50 simultaneous requests produced 49 errors. Adding a per-bay advisory lock fixed it. The k6 race now gives exactly 4 bookings for 4 bays and 46 clean `409 Slot unavailable` responses. See [ADR-001](#).
- 2. Events that are never lost or invented.** A transactional outbox writes each event in the same database transaction as the change it describes. A relay publishes it to Kafka, and every consumer skips duplicates. See [ADR-002](#).
- 3. A parts saga with compensation.** Stock is reserved all-or-nothing before work continues, consumed when the job completes, and released if it's cancelled. Row locks are taken in a fixed order so two jobs can't deadlock over the same parts.
- 4. Safe retries.** Booking takes an `Idempotency-Key`, so a double-click or a retry after a timeout returns the original booking.
- 5. Fast reads.** Availability is cached in Redis and falls back to PostgreSQL if Redis is down. It measures **p95 $\approx$ 3 ms at 200 req/s** locally.
- 6. AI-assisted, human-verified.** The design docs were drafted with an AI assistant and then checked. The [AI review log](#) records what it got wrong and how each mistake was caught.

Testing

Suite	Count	Command
Backend unit + integration (JUnit 5, Mockito, MockMvc, Testcontainers)	59	<code>make test-backend</code>
Frontend unit + component (Vitest, React Testing Library)	29	<code>make test-web</code>
End-to-end API walkthrough	13 steps	<code>make demo</code>
Load and race test (k6)	2 scenarios	<code>make loadtest</code>

`make test` runs both suites. Coverage reports go to `*/target/site/jacoco/index.html` (backend) and `web/coverage/index.html` (`make coverage-web`). Details are in the [testing guide](#).

Project structure

torqline/	
├─ torqline-common/	shared event contracts, outbox, idempotent consumer, error format
├─ gateway/	Spring Cloud Gateway
├─ appointment-service/	dealers, bays, availability, booking
├─ repair-order-service/	job cards, workflow, invoice, parts saga
├─ parts-inventory-service/	stock, reservations, low-stock alerts
├─ notification-service/	SMS and email from events
├─ web/	React + TypeScript UI (Vite, Vitest)
├─ docs/	design docs, guides, ADRs, article, screenshots
├─ loadtest/	k6 script
├─ scripts/	end-to-end demo
├─ docker-compose.yml	the whole stack
├─ Dockerfile	one image recipe for every Java service
└─ Makefile	everyday commands

Documentation

Document	What's in it
User guide	How to use each screen, with walkthroughs
API reference	Every endpoint with example requests and responses
High-level design	Problem, architecture, flows, non-functional requirements, trade-offs
Low-level design	Schemas, state machines, concurrency, messaging, caching
Testing guide	Test strategy, how to run each suite, what each test proves
Architecture decisions	Why the exclusion constraint, the outbox and database-per-service
AI review log	What AI suggested, what was accepted or rejected, and the bugs it caused
Article	The story of building it, written as a blog post
Interview guide	Likely interview questions with answers
Everything as one PDF	All of the above in a single printable file

Troubleshooting

Problem	Fix
A port is already in use (5433, 6379, 9092, 3000, 8080-8084)	Stop whatever uses it, or change the left-hand port in <code>docker-compose.yml</code>
Services restart repeatedly	Give Docker Desktop at least 6 GB of memory (Settings → Resources)
The UI looks outdated after a rebuild	Hard reload with Cmd+Shift+R
"No free slots"	The seeded days fill up after lots of testing. <code>make reset</code> starts fresh
The printout has a date and URL on it	In Chrome's print dialog, untick More settings → Headers and footers

Roadmap

- Kubernetes manifests / Helm chart and an AWS deployment (EKS, RDS, MSK, ElastiCache)
- Grafana dashboards for booking rate, conflicts, outbox lag and consumer lag
- Sign-in with customer and service-advisor roles; rate limiting at the gateway
- Reject a reused `Idempotency-Key` that comes with a different request body
- No-show handling and automatic slot release

Torqline: High-Level Design

1. Problem

A multi-brand service centre in Bengaluru works on **cars and bikes** in the same building, but a car lift and a bike stand are not interchangeable. Customers book online, walk in at their slot, and the workshop opens a job card (repair order), fits parts from the dealer's stock and invoices with GST.

Things that go wrong in real workshops, which this design targets:

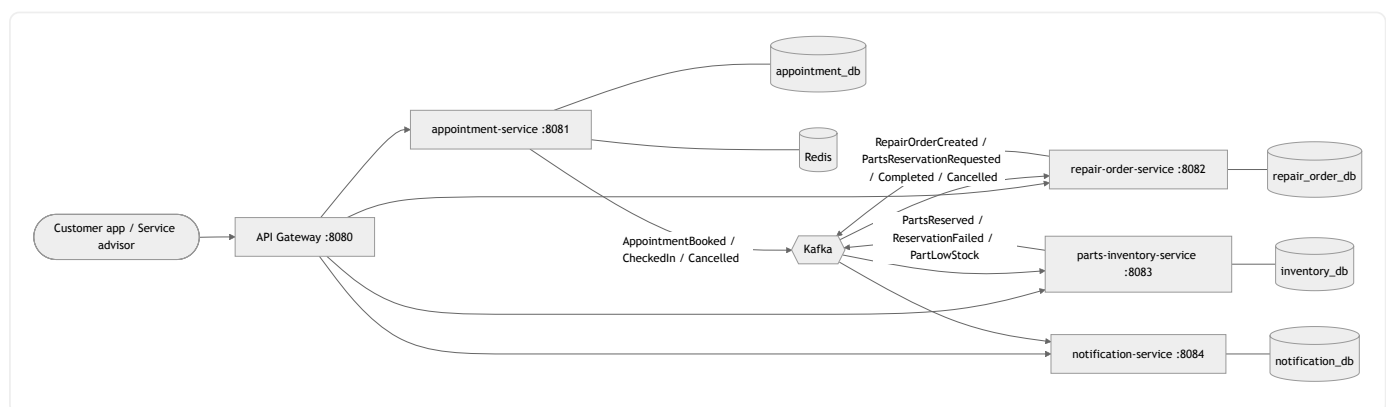
Problem	Consequence	Design response
Two customers get the same bay at the same time	Angry customers, idle technicians	Database-enforced no-overlap rule per bay
App retries a booking after a timeout	Duplicate appointments	Idempotency-Key on booking
A job card promises a part that is not on the shelf	Vehicle stuck on the lift	Parts reserved before work continues (saga)
Booking saved but "confirmed" SMS never sent (or vice versa)	Lost trust	Transactional outbox
Availability page hammered at 9 AM	Slow site, DB load	Redis read-through cache

2. Scope

In: dealers and bays, slot availability, booking/cancel/check-in, repair-order lifecycle, parts reservation/consumption, low-stock alerts, customer SMS/email (mocked).

Out (for now): authentication, payments, OEM warranty claims, technician scheduling, real SMS gateway.

3. Architecture

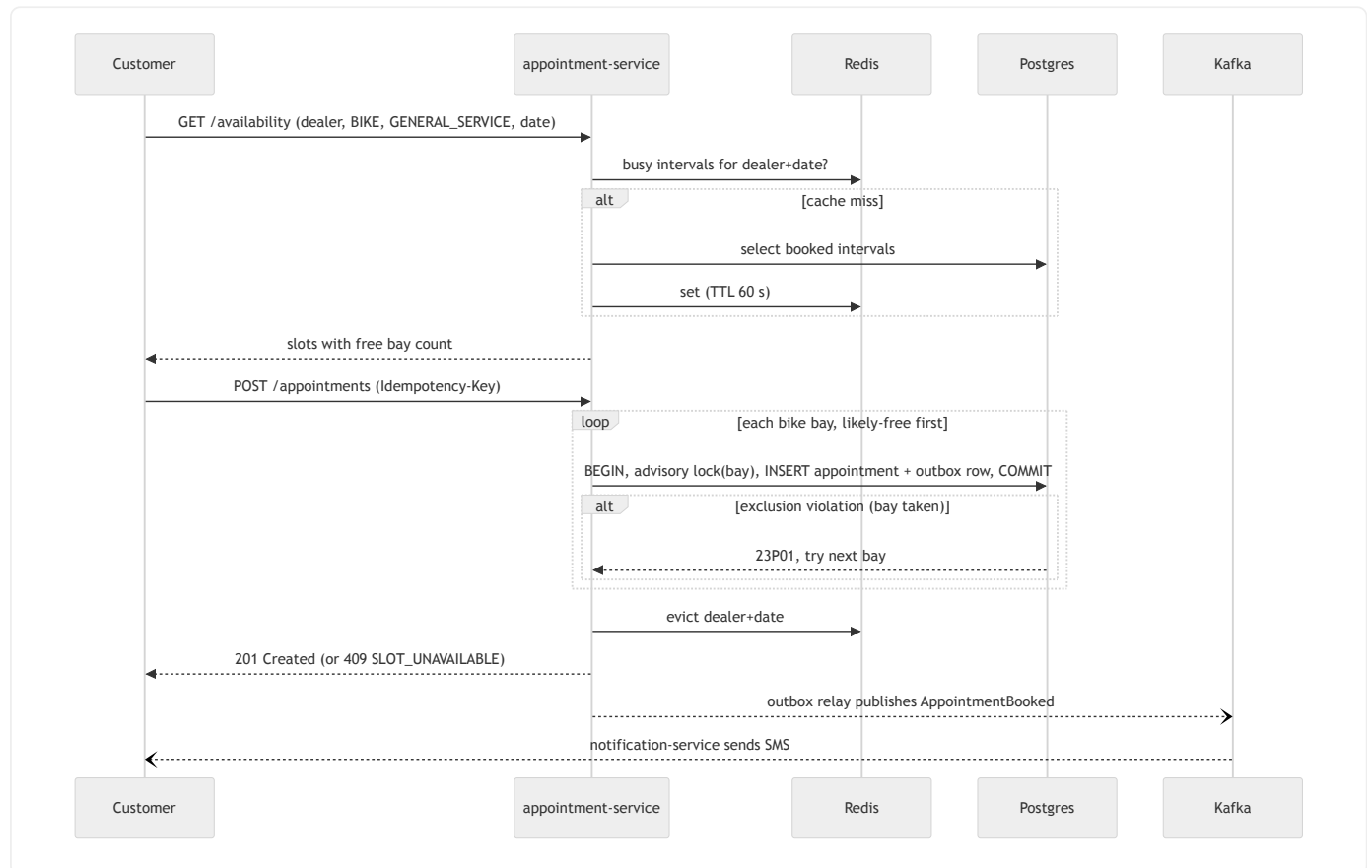


Service	Owns	Talks to others via
gateway	Routing, request ids	HTTP
appointment-service	Dealers, bays, appointments	Publishes appointment events
repair-order-service	Job cards, part lines, invoice totals	Consumes check-ins and inventory replies; publishes RO events
parts-inventory-service	Stock per dealer and SKU, reservations	Consumes RO events; publishes inventory events
notification-service	Sent-message log	Consumes everything, publishes nothing

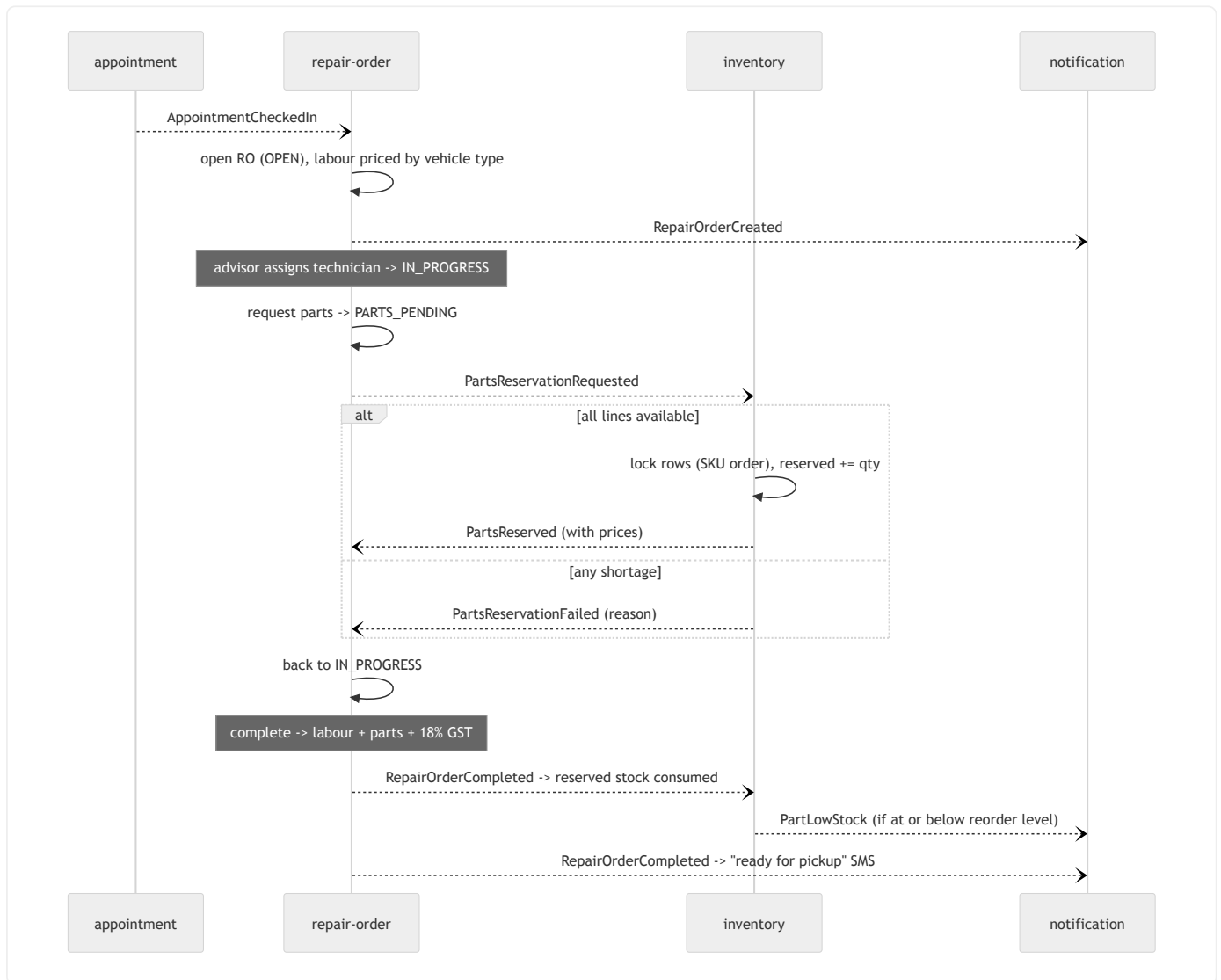
Database per service. Each service has its own Postgres database (one container locally). No service reads another's tables; the only coupling is the event contracts in `torqline-common`.

4. Key flows

4.1 Booking a slot



4.2 Check-in to invoice (parts saga)



Cancelling a repair order publishes `RepairOrderCancelled`; inventory **releases** the reservation (compensating action). Because RO events are keyed by repair-order id, request, complete and cancel for one order arrive in order on one partition.

5. Non-functional requirements

Concern	Target / approach
Correctness under concurrency	Zero double-bookings. Enforced by Postgres, verified by a 50-thread integration test and a k6 race
Availability latency	p95 < 150 ms for the availability endpoint; measured ~3 ms p95 at 200 req/s locally
Delivery guarantees	At-least-once events (outbox); exactly-once <i>effects</i> (idempotent consumers)
Failure isolation	Redis down means availability is served from Postgres (slower, still correct); Kafka down means outbox rows wait and ship later
Poison messages	3 retries, 1 s apart, then routed to <code><topic>-dlt</code>
Horizontal scale	All services stateless; outbox relay uses <code>FOR UPDATE SKIP LOCKED</code> so N instances can relay together; Kafka topics have 3 partitions
Observability	Actuator health/readiness, Prometheus metrics at <code>/actuator/prometheus</code> , <code>X-Request-Id</code> from the gateway

6. Deployment

- **Local:** `docker compose` (see README). One image per service, built from one multi-stage Dockerfile.
- **Cloud (next step):** EKS or ECS on AWS; RDS Postgres (one instance, database per service), ElastiCache Redis, MSK Kafka. Readiness/liveness probes already exposed via Actuator.

7. Trade-offs and alternatives considered

Decision	Alternative	Why
Postgres exclusion constraint + advisory lock	Redis distributed lock (Redlock)	The DB is already the source of truth; the invariant holds even if code has a bug or Redis fails over. See ADR-001
Transactional outbox	Publish to Kafka after commit	Dual-write gap loses or phantom-publishes events. See ADR-002
Choreographed saga	Central orchestrator service	Only two participants; the repair order already owns the state. Revisit if the flow grows (payments, warranty)
Fixed 30-minute grid	Free-form start times	Matches how workshops plan; keeps availability cheap to compute
Database per service	Shared schema	Independent deploys and migrations. See ADR-003

Torqline: Low-Level Design

1. Module layout

torqline-common	shared contracts and plumbing, no business logic
domain/	VehicleType (labour rate), ServiceType (duration per vehicle type)
events/	event records + topic names
messaging/	OutboxWriter, OutboxRelay, IdempotencyGuard, InboundEvent
web/	ApiException + RFC 9457 problem responses
config/	Spring Boot auto-configuration that wires the above into every service
appointment-service	dealer/, appointment/, slot/ (pure availability logic + Redis cache)
repair-order-service	order/ (aggregate + state machine), messaging/
parts-inventory-service	part/, reservation/, messaging/
notification-service	templates, sender port, listener
gateway	Spring Cloud Gateway routes + request-id filter

2. Domain rules: cars vs bikes

ServiceType	Car	Bike
GENERAL_SERVICE	120 min	60 min
OIL_CHANGE	30	30
BRAKE_SERVICE	90	60
TYRE_REPLACEMENT	60	30
CLUTCH_OVERHAUL	180	90
WHEEL_ALIGNMENT	60	not offered
AC_SERVICE	90	not offered
CHAIN_SPROCKET	not offered	60

- A booking can only use a bay of the same `vehicleType`.
- Labour = duration x hourly rate (CAR Rs 800/h, BIKE Rs 400/h). Invoice = labour + reserved parts + 18% GST.
- Parts have a `fitment` of CAR, BIKE or UNIVERSAL (coolant, brake fluid).

3. Data model

appointment_db

```
dealer(id PK, name, city, timezone, open_time, close_time)
service_bay(id PK, dealer_id FK, name, vehicle_type CHECK IN (CAR,BIKE))
appointment(id uuid PK, dealer_id, bay_id FK, customer_*, vehicle_*, service_type,
            slot_start timestampz, slot_end timestampz, status, idempotency_key UNIQUE,
            odometer_km, cancel_reason, created_at, updated_at, version)
EXCLUDE USING gist (bay_id WITH =, tstzrange(slot_start, slot_end, '[') WITH &&)
WHERE (status <> 'CANCELLED')
```

The exclusion constraint is the core invariant: a bay can never hold two live, overlapping bookings. `[ )` ranges mean a 10:00-11:00 booking does not conflict with an 11:00-12:00 one. Cancelled rows drop out of the constraint, so cancelling frees the slot immediately.

repair_order_db

```
repair_order(id PK, ro_number UNIQUE (R0-<year>-<seq>), appointment_id UNIQUE, dealer_id, customer_*,
            vehicle_*, service_type, status, technician, note,
            labour_amount, parts_amount, tax_amount, total_amount, opened_at, closed_at, version)
part_line(id PK, repair_order_id FK, request_id, sku, name, quantity, unit_price, status)
```

`appointment_id UNIQUE` is a second line of defence against duplicate job cards if an event is redelivered after the idempotency record was lost.

inventory_db

```
part(id PK, dealer_id, sku, name, fitment, unit_price, on_hand, reserved, reorder_level, version)
  UNIQUE (dealer_id, sku), CHECK (reserved >= 0 AND reserved <= on_hand)
reservation(id PK, request_id UNIQUE, repair_order_id, dealer_id, status, created_at, settled_at)
reservation_line(reservation_id FK, sku, quantity, unit_price)
```

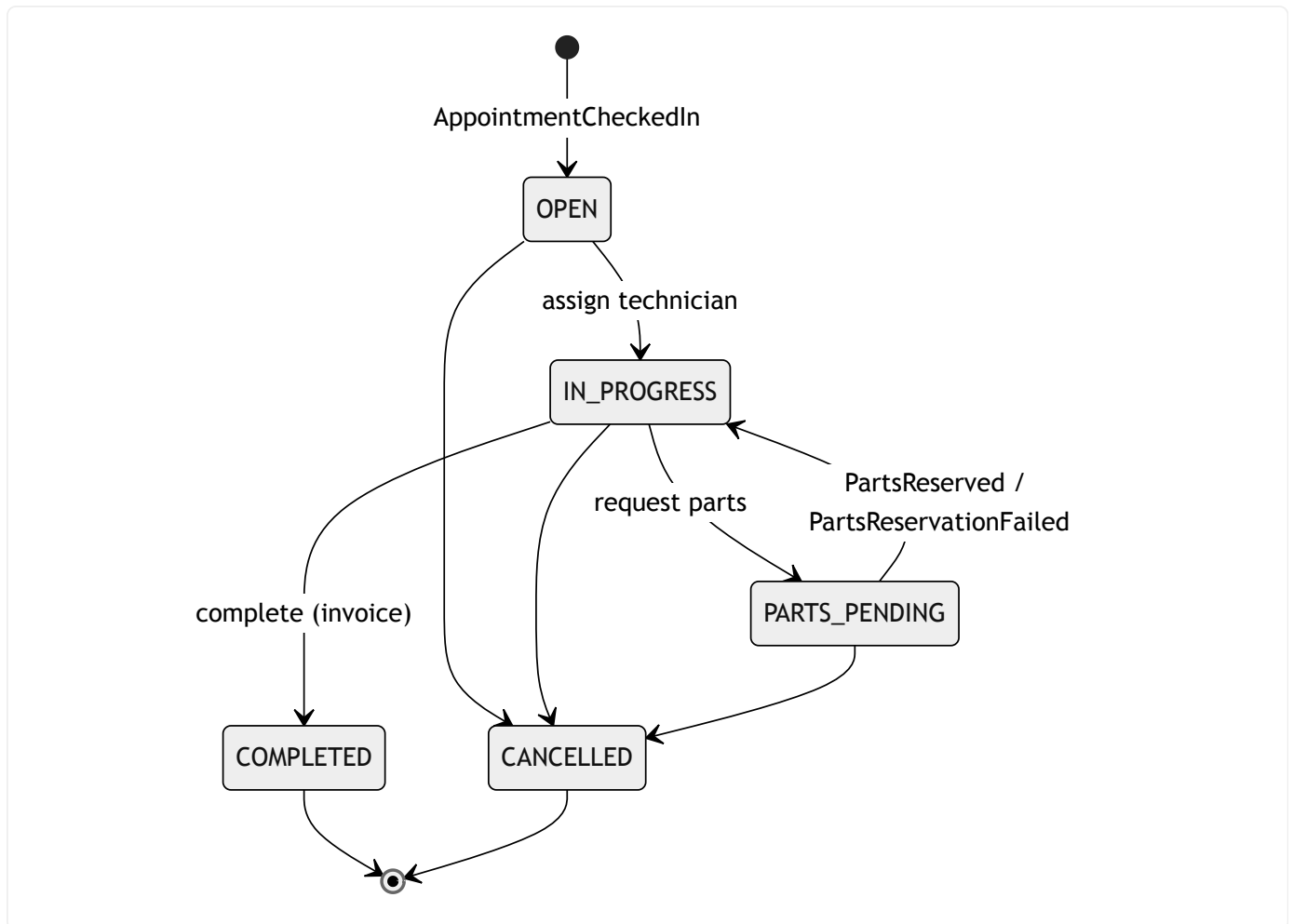
Every service

```
outbox_event(id PK, topic, aggregate_type, aggregate_id, event_type, payload jsonb, created_at,
            published_at)
processed_event(event_id PK, processed_at)
```

4. State machines

Appointment: BOOKED -> CHECKED_IN , BOOKED -> CANCELLED . Anything else returns 409 INVALID_STATE .

Repair order:



Transitions live in `RepairOrderStatus.next()`; the aggregate refuses anything else. An inventory reply for a request that is no longer pending (for example, the order was cancelled meanwhile) is ignored rather than reopening the order.

Reservation: `RESERVED -> CONSUMED` (RO completed) or `RESERVED -> RELEASED` (RO cancelled).

5. Concurrency

5.1 Booking

1. If an `Idempotency-Key` was already used, return that appointment with `200`.
2. Validate: job offered for vehicle type, on the 30-minute grid, inside opening hours, in the future.
3. Order candidate bays so the ones that look free (from the cached busy list) come first.
4. For each bay, in its **own short transaction**: `pg_advisory_xact_lock(7001, bay_id)`, then insert appointment and outbox row, then commit.
 - `23P01` exclusion violation: bay taken, try the next bay.
 - `23505` on the idempotency key: a concurrent retry won, return its result.
5. No bay left: `409 SLOT_UNAVAILABLE`.

Why the advisory lock: with the exclusion constraint alone, two transactions inserting overlapping rows for the same bay at the same instant each wait for the other's uncommitted row while checking the constraint, so Postgres detects a deadlock and aborts one after `deadlock_timeout` (1 s). Under the k6 race this turned 46 clean `409 s` into deadlock `500 s` and starved the connection pool. The lock serialises inserts **per bay only**. A transaction holds at most one such lock, so no cycle is possible. Correctness still comes from the constraint; the lock only removes the deadlock.

5.2 Parts reservation

`SELECT ... FOR UPDATE` on all requested SKUs, **ordered by SKU**, then check availability and update. Consistent lock order prevents deadlocks between two repair orders that need overlapping parts. The reservation is all-or-nothing: a shortage on any line reserves nothing and returns a reason listing every short line. The `ck_part_stock` CHECK is a final guard against overselling.

5.3 Optimistic locking

Aggregates carry `@Version`. Two advisors completing the same repair order at once get `409 CONCURRENT_MODIFICATION` instead of a silent lost update.

6. Messaging

Topic	Key	Events
<code>torqline.appointment.events</code>	appointment id	AppointmentBooked, AppointmentCancelled, AppointmentCheckedIn
<code>torqline.repair-order.events</code>	repair-order id	RepairOrderCreated, PartsReservationRequested, RepairOrderCompleted, RepairOrderCancelled
<code>torqline.inventory.events</code>	repair-order id or dealer:sku	PartsReserved, PartsReservationFailed, PartLowStock

- **Envelope:** the value is JSON; headers carry `eventId` (the outbox row id) and `eventType` (record name). Plain strings on the wire keep services independent of each other's class names.
- **Producer:** `OutboxWriter.append()` refuses to run outside a transaction. `OutboxRelay` polls every 500 ms, takes up to 100 rows with `FOR UPDATE SKIP LOCKED`, sends them, waits for all acks, then marks them published. Any failure rolls back the batch, which is retried later (at-least-once).
- **Consumer:** `IdempotencyGuard.processOnce(eventId, handler)` inserts into `processed_event` with `ON CONFLICT DO NOTHING` in the same transaction as the handler. A duplicate is skipped; if the handler fails, the marker rolls back too and the retry runs cleanly.
- **Errors:** `DefaultErrorHandler` with 3 retries then `DeadLetterPublishingRecoverer` to `<topic>-dlt`.

7. Caching

`BusySlotCache` stores a dealer's booked (`bayId`, `start`, `end`) intervals per day under `torqline:busy:<dealer>:<date>` for 60 s. Availability for any vehicle or service type is computed from that list plus the bay list, so one cache entry serves every query for that dealer and day, and one delete evicts it after a booking, cancellation or check-in.

Known staleness window: a reader that loaded from Postgres just before a booking committed can write the old list back after the eviction. The UI may then show a slot that is actually taken for up to the TTL; booking it returns `409`. This is acceptable because the cache never decides a booking.

Redis errors are logged and fall back to Postgres.

8. API

Method	Path	Notes
GET	/api/dealers , /api/dealers/{id} , /api/dealers/{id}/bays	
GET	/api/dealers/service-types	Durations per vehicle type
GET	/api/appointments/availability? dealerId&vehicleType&serviceType&date	
POST	/api/appointments	Idempotency-Key header; 201 new, 200 replay, 409 full
GET	/api/appointments?dealerId&date , /api/appointments/{id}	
POST	/api/appointments/{id}/check-in , /api/appointments/{id}/cancel	
GET	/api/repair-orders?dealerId[&status] , /api/repair-orders/{id}	
POST	/api/repair-orders/{id}/assign , /parts (202), /complete , /cancel	
GET	/api/parts?dealerId[&fitment] , /api/parts/low-stock , /api/parts/{dealer}/{sku}	
POST	/api/parts/{dealer}/{sku}/restock	
GET	/api/reservations?repairOrderId , /api/notifications[?recipient]	

Errors use RFC 9457 `ProblemDetail` with a stable `code` , for example:

```
{ "status": 409, "code": "SLOT_UNAVAILABLE", "detail": "No BIKE bay is free at 2026-09-27T10:00 for  
GENERAL_SERVICE" }
```

9. Testing

See [TESTING.md](#) for the full list. In short: 59 backend tests (unit, MockMvc, and Testcontainers race tests against real PostgreSQL), 29 frontend tests (Vitest + React Testing Library), a 13-step end-to-end script, and a k6 race and throughput test.

10. Web UI

React 19 + TypeScript, built with Vite and served by nginx, which also proxies `/api` to the gateway so there's no CORS. There's no client-side state library: each screen polls the API (1.5-3 s) while the tab is visible, which is enough to watch asynchronous saga steps land. `index.html` is served with `Cache-Control: no-cache` and hashed assets as immutable, so a rebuild is picked up on the next reload. Invoices print from a dedicated `#/invoice/<id>` page rather than from a dialog.

ADR-001: Prevent double booking in Postgres

Status: Accepted

Context

Bookings can arrive at the same moment from many customers and from several instances of appointment-service. A bay must never hold two overlapping live bookings.

Options

1. **Application check then insert:** a race window between check and insert.
2. **Redis lock per slot:** extra moving part; can lose the lock on failover or TTL expiry; still needs a DB check.
3. `SELECT ... FOR UPDATE` **on the bay row:** works, but serialises all reads of the bay, and the rule lives only in code.
4. **Postgres exclusion constraint** on `(bay_id WITH =, tstzrange(slot_start, slot_end) WITH &&) WHERE status <> 'CANCELLED'` .

Decision

Option 4, with a per-bay `pg_advisory_xact_lock` taken just before the insert.

Consequences

- The rule holds for every writer: bugs, scripts, a second service instance.
- Losing a race is a normal, fast `23P01` that the service turns into "try the next bay" or `409` .
- The advisory lock is needed because concurrent inserts that conflict under an exclusion constraint can deadlock (found under load; see AI-REVIEW #2). It serialises inserts per bay only; throughput across bays and dealers is unaffected.
- Requires the `btree_gist` extension (available on RDS/Aurora Postgres).

ADR-002: Transactional outbox for domain events

Status: Accepted

Context

State changes (booking, check-in, completion) must reliably produce events for other services. Writing to Postgres and Kafka separately is a dual write: either can succeed without the other.

Decision

Write events to an `outbox_event` table in the same transaction as the state change. A relay in each service polls unpublished rows (`FOR UPDATE SKIP LOCKED` , batch of 100), publishes them, waits for acks, then marks them published. Consumers deduplicate on `eventId` via `processed_event` .

Consequences

- An event exists if and only if the state change committed.
- Delivery is at-least-once, so every consumer must be idempotent (provided by `IdempotencyGuard`).
- About 0.5 s extra latency (poll interval). CDC with Debezium would cut this and remove polling; it is not worth the operational cost at this scale.
- Published rows are purged after 7 days.

ADR-003: Database per service

Status: Accepted

Context

Services need to deploy and change schema independently.

Decision

Each service owns one Postgres database and is the only one that reads or writes it. Locally, all four databases share one Postgres container; in the cloud they can share one RDS instance or be split.

Consequences

- No cross-service joins. Data another service needs travels in events (for example, the repair order keeps its own copy of customer and vehicle details from `AppointmentCheckedIn`).
- Cross-service consistency is eventual and handled with sagas and compensation.
- Reporting across services would need a read model or warehouse (out of scope).

Torqline API reference

All endpoints are reachable through the gateway at `http://localhost:8080/api`. Each service also serves interactive Swagger UI:

Service	Swagger UI
appointment-service	http://localhost:8081/swagger-ui.html
repair-order-service	http://localhost:8082/swagger-ui.html
parts-inventory-service	http://localhost:8083/swagger-ui.html
notification-service	http://localhost:8084/swagger-ui.html

Ready-made requests for IntelliJ and VS Code are in [api.http](#).

Conventions

- JSON in and out. Times in requests are the **dealer's local time** (`2030-01-07T10:00`); responses include both local times and UTC instants.
- Every response through the gateway carries an `X-Request-Id` header. Send your own to correlate logs.
- Errors use [RFC 9457](#) problem details with a stable `code`:

```
{
  "status": 409,
  "title": "Conflict",
  "code": "SLOT_UNAVAILABLE",
  "detail": "No BIKE bay is free at 2030-01-07T10:00 for GENERAL_SERVICE",
  "instance": "/api/appointments"
}
```

Code	Status	Meaning
VALIDATION_FAILED	400	A field is missing or malformed; <code>errors</code> maps each field to a message
OFF_GRID	400	Slot doesn't start on the hour or half hour
OUTSIDE_HOURS	400	Job wouldn't fit inside opening hours
SLOT_IN_PAST	400	Slot has already started
SERVICE_NOT_OFFERED	400	Job doesn't apply to the vehicle type, e.g. wheel alignment for a bike
NO_BAYS	400	Branch has no bays for that vehicle type
NOT_FOUND	404	Unknown id
SLOT_UNAVAILABLE	409	Every suitable bay is taken for that time
INVALID_STATE / INVALID_TRANSITION	409	Action not allowed in the current status
CONCURRENT_MODIFICATION	409	Someone else changed the record; retry

Dealers

GET /dealers

```
[{ "id": "TQ-BLR-IND", "name": "Torqline Indiranagar", "city": "Bengaluru", "timezone": "Asia/Kolkata",  
  "openTime": "09:00:00", "closeTime": "18:00:00", "bayCount": { "CAR": 3, "BIKE": 2 } }]
```

GET /dealers/{dealerId} • GET /dealers/{dealerId}/bays

GET /dealers/service-types

The service catalogue with duration per vehicle type. A missing key means the job isn't offered for that vehicle.

```
[{ "code": "GENERAL_SERVICE", "description": "Periodic maintenance service", "durationMinutes": { "CAR":  
120, "BIKE": 60 } },  
{ "code": "CHAIN_SPROCKET", "description": "Chain and sprocket kit replacement", "durationMinutes": {  
  "BIKE": 60 } }]
```

Appointments

GET /appointments/availability?dealerId&vehicleType&serviceType&date

```
{ "dealerId": "TQ-BLR-IND", "date": "2030-01-07", "vehicleType": "BIKE", "serviceType": "GENERAL_SERVICE",  
  "durationMinutes": 60,  
  "slots": [ { "start": "09:00:00", "end": "10:00:00", "freeBays": 1 },  
    { "start": "09:30:00", "end": "10:30:00", "freeBays": 0 } ] }
```

POST /appointments

Header `Idempotency-Key: <uuid>` is optional but recommended.

```
{ "dealerId": "TQ-BLR-IND", "customerName": "Asha Rao", "customerPhone": "9845012345",  
  "vehicleType": "BIKE", "vehicleNumber": "KA 03 HB 1234", "vehicleMake": "Royal Enfield",  
  "vehicleModel": "Classic 350", "serviceType": "GENERAL_SERVICE", "slotStart": "2030-01-07T10:00",  
  "notes": "Front brake squeals" }
```

Response	When
201 Created	New booking
200 OK	Same <code>Idempotency-Key</code> seen before; returns the original booking
409 SLOT_UNAVAILABLE	Every bay of that vehicle type is taken

```
{ "id": "de63d0bd-...", "dealerId": "TQ-BLR-IND", "bayId": 4, "status": "BOOKED",  
  "vehicleType": "BIKE", "vehicleNumber": "KA03HB1234", "serviceType": "GENERAL_SERVICE",  
  "localStart": "2030-01-07T10:00:00", "localEnd": "2030-01-07T11:00:00",  
  "slotStart": "2030-01-07T04:30:00Z", "slotEnd": "2030-01-07T05:30:00Z", "...": "..." }
```

GET /appointments?dealerId&date • GET /appointments/{id}

POST /appointments/{id}/check-in

Body (optional): { "odometerKm": 18500 }. Moves `BOOKED` → `CHECKED_IN` and opens a repair order asynchronously.

POST /appointments/{id}/cancel

Body: { "reason": "Customer travelling" }. Moves BOOKED → CANCELLED and frees the bay.

Repair orders

GET /repair-orders?dealerId[&status] • GET /repair-orders/{id}

```
{ "id": "dfbb355a-...", "roNumber": "R0-2026-001002", "status": "COMPLETED", "vehicleType": "BIKE",  
  "vehicleNumber": "KA05TR4242", "serviceType": "GENERAL_SERVICE", "technician": "Ravi K",  
  "parts": [ { "sku": "BRAKE-PAD-BIKE", "name": "Disc brake pad set (bike)", "quantity": 1,  
               "unitPrice": 650.00, "status": "RESERVED" } ],  
  "labourAmount": 400.00, "partsAmount": 1100.00, "taxAmount": 270.00, "totalAmount": 1770.00 }
```

POST /repair-orders/{id}/assign

{ "technician": "Ravi K" } • OPEN → IN_PROGRESS

POST /repair-orders/{id}/parts → 202 Accepted

{ "lines": [{ "sku": "OIL-10W30-1L", "quantity": 1 }] } • IN_PROGRESS → PARTS_PENDING . Poll the order: it returns to IN_PROGRESS with each line RESERVED , or all lines REJECTED and a note explaining why.

POST /repair-orders/{id}/complete

IN_PROGRESS → COMPLETED ; fills in partsAmount , taxAmount and totalAmount .

POST /repair-orders/{id}/cancel

{ "reason": "..." } . Any reserved parts are released.

Parts and reservations

Endpoint	Description
GET /parts?dealerId[&fitment=CAR BIKE UNIVERSAL]	Stock list; CAR and BIKE also include universal parts
GET /parts/low-stock?dealerId	Parts at or below the reorder level
GET /parts/{dealerId}/{sku}	One part
POST /parts/{dealerId}/{sku}/restock	{ "quantity": 10 }
GET /reservations?repairOrderId	Reservations held for a job

Notifications

GET /notifications[?recipient][&limit=50] : sent messages, newest first.

Events (Kafka)

Services also communicate through these topics. Payloads are JSON; eventId and eventType travel as headers.

Topic	Events
torqline.appointment.events	AppointmentBooked , AppointmentCancelled , AppointmentCheckedIn
torqline.repair-order.events	RepairOrderCreated , PartsReservationRequested , RepairOrderCompleted , RepairOrderCancelled
torqline.inventory.events	PartsReserved , PartsReservationFailed , PartLowStock

Messages that keep failing are moved to `<topic>-dlt` after 3 retries. Browse topics with `make tools` → <http://localhost:8090>.

Testing guide

Layer	Tools	Count	Runs in
Backend unit	JUnit 5, AssertJ, Mockito	49	milliseconds, no Docker
Backend web layer	MockMvc (standalone)	5	milliseconds
Backend integration	Spring Boot test + Testcontainers PostgreSQL	5	~10 s, needs Docker
Frontend unit and component	Vitest, React Testing Library, jsdom	29	~3 s
End-to-end	<code>scripts/demo.sh</code> against the running stack	13 steps	~10 s
Load and race	k6	2 scenarios	~25 s

Running

```
make test           # backend + frontend
make test-backend  # ./mvnw verify, or Maven in Docker if you have no JDK
make test-web      # npm test, or Node in Docker if you have no Node
make coverage-web  # web/coverage/index.html
make demo          # needs `make up`
make loadtest      # needs `make up`
```

Backend coverage reports (JaCoCo) are written to `<module>/target/site/jacoco/index.html` by `make test-backend`.

Single tests:

```
./mvnw -pl appointment-service -am test -Dtest=BookingConcurrencyTest -
Dsurrefire.failIfNoSpecifiedTests=false
cd web && npx vitest run src/pages/BookPage.test.tsx
```

What each backend test proves

Test	Proves
<code>ServiceTypeTest</code>	Car and bike durations differ; vehicle-only jobs are rejected for the other type; every job fits the 30-minute grid
<code>InboundEventTest</code>	Kafka envelope headers decode; records without an envelope are rejected
<code>OutboxWriterTest</code>	Events can't be written outside the business transaction
<code>SlotGridTest</code>	Availability maths: last slot ends at closing, overlaps reduce free bays, other vehicle types' bookings are ignored, past slots are hidden
<code>AppointmentServiceTest</code>	Booking validation (grid, opening hours, past, vehicle fit, unknown dealer, no bays); idempotent replay doesn't insert; only booked appointments can be checked in or cancelled
<code>AppointmentControllerTest</code>	201 new vs 200 replay; 409 carries <code>SLOT_UNAVAILABLE</code> ; invalid input is a 400 with per-field errors and never reaches the service
<code>BookingConcurrencyTest</code>	Against real PostgreSQL: 50 threads racing for 2 bays → exactly 2 bookings; 50 threads for 4 bays → exactly 4, with no deadlocks ; idempotent retry; cancelling frees the bay
<code>RepairOrderTest</code>	Labour pricing per vehicle; invoice with 18% GST; can't complete while parts are pending; rejected parts aren't charged; late inventory replies after cancel are ignored
<code>RepairOrderServiceTest</code>	RO numbering; no duplicate job card per appointment; parts requests merge and normalise SKUs; completion and cancellation publish the right events
<code>PartTest</code>	Reserve, release and consume arithmetic; can't over-reserve; low-stock threshold
<code>InventoryServiceTest</code>	All-or-nothing reservation; shortage and unknown-SKU reasons; duplicate lines summed; redelivered request ignored; consumption raises low-stock alerts; release returns stock
<code>MessageTemplatesTest</code> , <code>NotificationServiceTest</code>	Message wording and IST times; SMS to customer, email to manager; internal events notify nobody

What each frontend test proves

Test	Proves
<code>format.test.ts</code>	Indian rupee formatting (₹1,25,000.00), labels, 12-hour times, day strip
<code>api.test.ts</code>	<code>Idempotency-Key</code> is sent; problem responses become <code>ApiError</code> with code and message; field errors are listed; query strings
<code>InvoiceView.test.tsx</code>	Subtotal, GST and total; rejected parts left off the bill
<code>BookPage.test.tsx</code>	Services filtered by vehicle; full slots disabled; booking sends an idempotency key and shows the confirmation; a 409 shows the error and refreshes slots; confirm disabled until a slot is picked
<code>WorkshopPage.test.tsx</code>	Cancelled appointments hidden; cards in the right column; check-in sends the odometer; assign sends the chosen technician; server errors are shown
<code>InventoryPage.test.tsx</code>	Low-stock warning; reserved quantities; fitment filter; restock
<code>InvoicePage.test.tsx</code>	Printable page renders and prints only when asked; warns on unfinished jobs; unknown order shows an error
<code>ActivityDrawer.test.tsx</code>	Lists SMS and email with the triggering event; empty state

Bugs the tests caught

These are real, and each is written up in the [AI review log](#):

1. **Deadlock under contention.** The k6 race turned 46 expected 409 s into 500 s. `BookingConcurrencyTest` was made harder until it failed the same way, then the fix (per-bay advisory lock) was verified against it.
2. **A test that could never fail.** The first k6 script gave each virtual user its own random date, so nothing actually raced, yet it "passed" with 50 of 50 booked.
3. **Validation errors without details.** `AppointmentControllerTest` showed that a bad phone number returned a bare 400 without saying which field was wrong, because Spring 7 raises a different exception when a handler also validates a header. The error handler now covers both.

Torqline user guide

Torqline has three screens, one per role, plus a live message feed. Switch screens with the tabs at the top, and switch branch with the dropdown in the top-right corner.

Open the app at <http://localhost:3000> after `make up`.

Seeded data

Branch	Car lifts	Bike stands	Hours (IST)	Notes
Torqline Indiranagar (TQ-BLR-IND)	3	2	09:00-18:00	Car-heavy; only 1 chain kit in stock
Torqline Whitefield (TQ-BLR-WHF)	2	4	09:00-19:00	Bike-heavy

Each branch stocks 16 parts: 7 for cars, 7 for bikes and 2 universal (coolant, brake fluid).

1. Book service (customer)

Book service

Customer

Workshop

Service advisor

Inventory

Parts desk

Torqline Indiranagar

1 Your vehicle

Bike / scooter

2 stands at this branch

Car

3 lifts at this branch

2 Service

General service

Periodic maintenance service

60 MIN

Oil change

Engine oil and filter change

30 MIN

Brake service

Brake inspection and pad replacement

60 MIN

Tyre replacement

Tyre replacement and balancing

30 MIN

Clutch overhaul

Clutch overhaul

90 MIN

Chain sprocket

Chain and sprocket kit replacement

60 MIN

3 Date and time

Today 26 Sept

Tomorrow 27 Sept

Mon 28 Sept

Tue 29 Sept

Wed 30 Sept

Thu 1 Oct

Fri 2 Oct

Sat 3 Oct

9:00 AM

9:30 AM

10:00 AM

10:30 AM

11:00 AM

11:30 AM

12:00 PM

12:30 PM

1:00 PM

1:30 PM

2:00 PM

2:30 PM

3:00 PM

3:30 PM

4:00 PM

4:30 PM

5:00 PM

4 Your details

Name

Chethan S

Mobile

98450 22222

Registration number

KA 51 EZ 7788

Make

TVS

Model

Apache RTR 160

Notes

Front brake squeals

General service

27 Sept, 10:30 AM to 11:30 AM

Confirm booking

1. **Your vehicle:** choose *Bike / scooter* or *Car*. The card shows how many stands or lifts the branch has.

2. **Service:** only services that apply to your vehicle appear. A bike sees *Chain sprocket*; a car sees *Wheel alignment* and *AC service*. Each card shows how long the job takes for that vehicle.
3. **Date and time:** pick a day, then a slot. Each slot shows how many bays are free:
 - green, *2 of 2 free*: plenty of room
 - amber, *1 of 2 free*: last bay
 - grey, *Full*: can't be booked
4. **Your details:** name, 10-digit mobile number and registration number are required; make, model and notes are optional.
5. **Confirm booking.** You get a reference number, the bay, and an SMS (open the 🛎 to see it).

What happens behind the scenes

- If someone else takes the last bay while you're filling in the form, you'll see *No BIKE bay is free at ...* and the slot grid refreshes. You can never be double-booked.
- Pressing **Confirm** twice, or a retry after a network timeout, doesn't create two bookings. Each attempt carries an idempotency key and the server returns the original booking.

2. Workshop (service advisor)

 **Torqline**

Book service
Customer

Workshop
Service advisor

Inventory
Parts desk

Torqline Indiranagar



Appointments

Check a vehicle in when it arrives. That opens a job card on the board below.

Today
26 Sept

**Tomorrow
27 Sept**

Mon
28 Sept

Tue
29 Sept

Wed
30 Sept

Thu
1 Oct

Fri
2 Oct

Sat
3 Oct

TIME	VEHICLE	CUSTOMER	SERVICE	STATUS
9:00 AM to 10:00 AM	 KA03HB1234 Royal Enfield Classic 350	Asha Rao 9845026208	General service	Checked in
9:00 AM to 10:30 AM	 KA01MJ4321 Maruti Suzuki Baleno	Asha Rao 9845026208	AC service	Checked in
10:00 AM to 11:00 AM	 KA05TR4242 Bajaj Pulsar NS200	Test Rider 9845011111	General service	Checked in

Workshop board

Updates live. Parts requests are handled asynchronously by the inventory service over Kafka.

● **Checked in**

Waiting for a technician

0

● **In progress**

On the lift or stand

1

R0-2026-001003

 Car

KA01MJ4321

Maruti Suzuki Baleno

AC service

Technician **Ravi K**

Add parts

Complete

● **Waiting for parts**

Inventory is reserving stock

0

● **Ready for pickup**

Invoiced

2

R0-2026-001002

 Bike

KA05TR4242

Bajaj Pulsar NS200

General service - 12,400 km

Technician **Ravi K**

1x Disc brake pad set (bike) ₹650.00

1x Engine oil 10W-30, 1 L ₹450.00

Invoice - ₹1,770.00

R0-2026-001001

 Bike

KA03HB1234

Royal Enfield Classic 350

General service - 18,500 km

Technician **Ravi K**

1x Brake fluid DOT 4, 500 ml ₹420.00

2x CHAIN-KIT Rejected

1x Engine oil 10W-30, 1 L ₹450.00

1x Spark plug (bike) ₹150.00

Parts request rejected: CHAIN-KIT needs 2, only 1 available

Invoice - ₹1,675.60

Torqline · 37 / 47

Appointments

Pick a day to see its appointments. For a vehicle that has arrived:

- type the **odometer** reading (optional) and click **Check in**, or
- click **Cancel** if the customer doesn't turn up. That frees the bay immediately.

Checking in sends an event to the repair-order service, which opens a **job card** on the board within a second or two.

Workshop board

Column	Meaning	Actions
Checked in	Job card open, nobody assigned	Choose a technician → Start work
In progress	On the lift or stand	Add parts, Complete
Waiting for parts	Inventory is reserving stock	None; wait for it to move back automatically
Ready for pickup	Completed and invoiced	Invoice

Adding parts: the picker lists only parts that fit the vehicle, plus universal ones, with live stock. Choose quantities with – / + and click **Request parts**. The card moves to *Waiting for parts* and then back to *In progress*:

- **Reserved** lines show in green with their price.
- If *any* line is short, the **whole request is rejected** (struck through in red) with a reason, for example *CHAIN-KIT needs 2, only 1 available*. Nothing is reserved, so you can request again with different quantities.

Completing prices the job:

```
labour  = job duration × labour rate (car ₹800/h, bike ₹400/h)
parts   = sum of reserved lines
GST     = 18% of (labour + parts)
total   = labour + parts + GST
```

Rejected parts are never charged.

Invoice and printing

Invoice shows the itemised bill. **Print** opens a clean, printable invoice page in a new tab and brings up the print dialog. Choose *Save as PDF* to keep a copy. To remove the date and URL that Chrome adds, untick **More settings** → **Headers and footers**.

[← Back to workshop](#)[Print / Save as PDF](#)**TAX INVOICE**

RO-2026-001002 · 26 Sept 2026, 9:37 pm

Torqline Indiranagar

Bengaluru

Test Rider

9845011111

Bike KA05TR4242 · Bajaj Pulsar NS200 · Technician Ravi K

ITEM	QTY	RATE	AMOUNT
Labour: General service	1	₹400.00	₹400.00
Disc brake pad set (bike)	1	₹650.00	₹650.00
Engine oil 10W-30, 1 L	1	₹450.00	₹450.00
Subtotal			₹1,500.00
GST 18%			₹270.00
Total			₹1,770.00

Thank you for servicing with Torqline. This is a computer-generated invoice.

3. Inventory (parts desk)

 **Torqline**

[Book service](#)
Customer

[Workshop](#)
Service advisor

[Inventory](#)
Parts desk

Torqline Indiranagar 

**Parts inventory**

Reserved parts are promised to open job cards. They leave the shelf when the job completes, or return to stock if it's cancelled.

All Car Bike Universal

2 part(s) at or below reorder level. The service manager has been emailed.

- **Filter** by Car, Bike or Universal.
- The **stock bar** shows available (green) and reserved (amber) stock, with the reorder level as a red marker.

- **Reserved** stock is promised to open job cards. It leaves the shelf when the job completes and returns if the job is cancelled.
- Rows at or below the reorder level are highlighted, and the service manager is emailed automatically.
- **+10** restocks a part.

4. Messages (🔔)

[Book service](#)
Customer

[Workshop](#)
Service advisor

[Inventory](#)
Parts desk

Torqline Indiranagar 🔔

Parts inventory

Reserved parts are promised to open job cards. They leave the shelf when the job completes, or return to stock if it's cancelled.

2 part(s) at or below reorder level. The service manager has been emailed.

PART	FITS	PRICE	STOCK
AC refrigerant R134a, 500 g AC-GAS-R134A	CAR	₹1,500.00	20 on hand
Air filter (bike) AIR-FILTER-BIKE	BIKE	₹320.00	8 on hand
Engine air filter (car) AIR-FILTER-CAR	CAR	₹650.00	20 on hand
Brake fluid DOT 4, 500 ml BRAKE-FLUID-DOT4	UNIVERSAL	₹420.00	14 on hand
Disc brake pad set (bike) BRAKE-PAD-BIKE	BIKE	₹650.00	7 on hand
Front brake pad set (car) BRAKE-PAD-CAR-F	CAR	₹2,400.00	20 on hand
Cabin AC filter CABIN-FILTER	CAR	₹550.00	20 on hand
Chain and sprocket kit CHAIN-KIT	BIKE	₹2,200.00	11 on hand
Clutch plate set (bike) CLUTCH-PLATE-BIKE	BIKE	₹1,400.00	8 on hand

Messages sent

SMS and email from notification-service

SMS

9845026208

19m ago

Job card RO-2026-001003 opened for KA01MJ4321. Track progress with this number.

Repair order created

SMS

9845026208

19m ago

Hi Asha Rao, we have received KA01MJ4321. We'll message you when the job card is open.

Appointment checked in

EMAIL

manager+TQ-BLR-IND@torqline.dev

19m ago

Low stock at TQ-BLR-IND: Engine oil 10W-30, 1 L (OIL-10W30-1L) has 6 available, reorder level 8.

Part low stock

SMS

9845011111

20m ago

Good news Test Rider! KA05TR4242 is ready for pickup. Invoice RO-2026-001002 total Rs 1770.00 (incl. GST).

Repair order completed

SMS

9845011111

20m ago

Job card RO-2026-001002 opened for KA05TR4242. Track progress with this number.

Repair order created

SMS

9845011111

20m ago

Hi Test Rider, we have received KA05TR4242. We'll message you when the job card is open.

Every message the notification service sent, newest first, updating live:

When	Channel	To
Appointment booked, cancelled, checked in	SMS	Customer
Job card opened, job completed (with invoice total)	SMS	Customer
Part at or below reorder level	Email	manager+<branch>@torqline.dev

Messages are logged rather than actually sent. Swapping in a real SMS gateway means implementing one interface (`NotificationSender`).

Walkthroughs

Race for the last bay: open *Book service* in two browser windows, pick the same bike slot at Indiranagar (2 stands), and confirm in both. Do it once more in a third window. The third booking is refused.

Stock-out: check in a bike at Indiranagar, start work, and request **2 × Chain and sprocket kit**. The request is rejected because only 1 is in stock. Go to **Inventory**, click **+10** on the chain kit, then request again. This time it's reserved.

Cancel with parts reserved: reserve parts on a job, then cancel it through the API (`POST /api/repair-orders/{id}/cancel`). On the Inventory screen the reserved quantity returns to available.

AI-assisted design and review log

This project was designed and built with an AI coding assistant. This log records what the AI proposed, how each proposal was checked, and the decision. The aim is to show judgement: which suggestions were accepted, which were changed, and which were wrong and how that was found.

Before sharing: rewrite these entries in your own words, and add your own as you extend the project. Be ready to explain each one in an interview. Entries 2 and 3 describe real failures from building this project.

#	Area	AI proposal	How it was checked	Verdict
1	Double booking	Redis distributed lock (<code>SETNX</code> per slot) around the booking	Asked what happens on Redis failover or lock-TTL expiry during a slow insert: the lock can be lost and two writers proceed. The DB is the source of truth anyway	Rejected. Postgres exclusion constraint on <code>(bay_id, tstzrange)</code> . ADR-001
2	Double booking	The exclusion constraint alone is enough under concurrency	k6 race: 50 users, 4 bays. 1 booked, 49 failed with HTTP 500 , and availability p95 jumped to 23 s. Postgres log: <code>deadlock detected ... while checking exclusion constraint</code> . The 20-thread test had passed because its 10-connection pool limited real parallelism	Changed. Added a per-bay <code>pg_advisory_xact_lock</code> . Raised the test to 50 threads with a 60-connection pool plus a 4-bay case; confirmed it fails without the lock and passes 3/3 with it. k6 afterwards: 4 booked, 46 clean 409s, p95 2.6 ms
3	Load test	k6 script picked a random date in init code	First run reported all 50 racers booked , which is impossible with 4 bays. k6 runs init code once per VU, so each VU raced on its own day	Fixed the test. Date chosen in <code>setup()</code> and shared. Lesson: a passing test can be wrong, so check the number actually makes sense
4	Events	Save the entity, then <code>kafkaTemplate.send()</code> in the same method	Walked through a crash between commit and send (event lost) and a send followed by rollback (phantom event)	Rejected. Transactional outbox + relay with <code>SKIP LOCKED</code> . ADR-002
5	Events	Exactly-once via Kafka transactions	Kafka transactions do not cover the Postgres write, so they don't remove the dual write	Rejected. At-least-once delivery + idempotent consumers (<code>processed_event</code>)
6	Caching	<code>@Cacheable</code> on the availability method keyed by (dealer, vehicle, service, date)	Eviction after a booking would need to clear every vehicle/service combination for that day	Changed. Cache the raw busy intervals per dealer-day. One key, one evict, availability computed in memory
7	Caching	Evict inside the booking transaction	Evicting before commit lets a concurrent reader repopulate with pre-commit data	Changed. Evict after commit. Remaining small race documented in LLD section 7; acceptable because the DB decides
8	Inventory	Check stock, then <code>UPDATE</code> per line	Two concurrent requests can both pass the check (check-then-act)	Changed. <code>SELECT ... FOR UPDATE</code> on all SKUs in SKU order (avoids deadlocks), plus a <code>CHECK</code> constraint <code>reserved <= on_hand</code>
9	Saga	Separate orchestrator service	Only two participants and the repair order already owns the state	Rejected for now. Choreography with RO state machine; revisit if payments or warranty join
10	Schema	<code>spring.jpa.hibernate.ddl-auto=update</code>	Unreviewable, can't express exclusion constraints, risky in prod	Rejected. Flyway migrations + <code>ddl-auto=validate</code>
11	Idempotency	Store key only	A client reusing a key with a <i>different</i> body silently gets the old booking	Accepted with known gap. Next step: store a request hash and return 422 on mismatch

#	Area	AI proposal	How it was checked	Verdict
12	Late replies	Apply <code>PartsReserved</code> whenever it arrives	Cancel then late reserve would reopen a cancelled job	Changed. Aggregate ignores replies not matching a pending request; unit-tested
13	Error handling	One <code>@ExceptionHandler</code> for <code>MethodArgumentNotValidException</code> covers all validation	A <code>MockMvc</code> test sent a bad phone number and got a bare 400 with no field details. With <code>@Size</code> on the <code>Idempotency-Key</code> header, Spring 7 runs method validation and throws <code>HandlerMethodValidationException</code> instead	Fixed. Handle both exceptions with one response shape; the controller test guards it
14	UI	Print the invoice by hiding the app with <code>@media print</code> while the dialog is open	The user's printout was blank: the dialog lived inside the hidden app. Moving it to a portal worked in the preview, but the browser was still serving a cached bundle	Changed. A dedicated <code>#/invoice/<id></code> page, verified by printing to PDF in headless Chrome; <code>Cache-Control: no-cache</code> on <code>index.html</code>

Prompts that worked well

- "List every way this can double-book, including across two instances of the service."
- "What happens if the process dies between line X and line Y?"
- "Write a test that fails if this guarantee is broken, and show me it failing first."
- "Is this number plausible?" (asked about the 50/50 booked result, which exposed the bad test)

Building a double-booking-proof service platform for car and bike workshops

How I designed, built and load-tested an event-driven dealership system in Java, and the two bugs that only showed up when I measured it.

Walk into any multi-brand service centre in Bengaluru on a Saturday morning and you'll see the same thing: a row of car lifts, a row of bike stands, a service advisor with a phone in each hand, and a whiteboard that's never quite right. Two customers were told "10 o'clock is fine". A job card says brake pads, but the last set went to another car an hour ago. Someone's bike was ready at noon, but nobody told them.

I wanted to build the software that makes those problems impossible rather than just unlikely. The result is **Torqline**: customers book a slot for a car or a bike, advisors run the workshop from a live job board, parts are reserved before work continues, and customers get SMS updates and an itemised GST invoice.

This article covers the design choices, the tools, and the part I learned the most from: the two bugs that every unit test passed but a load test exposed.

Cars and bikes are not the same customer

The first design decision was to model the two kinds of vehicle honestly instead of adding a `type` column and moving on:

- **Different bays.** A hatchback doesn't fit on a bike stand. Every bay has a vehicle type, and a booking can only land on a bay of the same type.
- **Different job lengths.** A general service is 2 hours for a car and 1 hour for a bike, so availability depends on the vehicle as well as the service.
- **Different jobs.** Bikes need chain and sprocket kits; cars need wheel alignment and AC service. The API refuses a wheel alignment for a scooter, and the UI never offers it.
- **Different money.** Labour is ₹800/hour for cars and ₹400/hour for bikes, and parts have a fitment of car, bike or universal.

All of this lives in two small Java enums shared by every service. That's the kind of domain knowledge that belongs in code rather than in a wiki.

The architecture

Torqline is five Spring Boot services behind a gateway, plus a React front end:

- **appointment-service:** dealers, bays, availability and bookings
- **repair-order-service:** job cards and their workflow
- **parts-inventory-service:** stock and reservations
- **notification-service:** SMS and email
- **gateway:** one entry point

Each service owns its own PostgreSQL database, and they coordinate through Kafka. When an advisor checks a vehicle in, the appointment service doesn't call the repair-order service. It records an `AppointmentCheckedIn` event, and the repair-order service reacts by opening a job card. When the job needs parts, a small **saga** runs: the repair order asks for stock, inventory either reserves all of it or none of it, and the order moves on. If the job is cancelled, inventory releases what it held.

Two patterns carry most of the reliability:

The transactional outbox. Saving a booking and publishing "booking confirmed" to Kafka are two separate writes, and either can fail on its own. You then either confirm a booking that doesn't exist or never confirm one that does. So each service writes its events to an `outbox_event` table **in the same transaction** as its data, and a relay publishes them afterwards using `SELECT ... FOR UPDATE SKIP LOCKED` so several instances can share the work.

Idempotent consumers. The relay can send an event twice (that's the price of never losing one), so every consumer records the event ID in the same transaction as its work and skips anything it has already seen.

The hardest problem: two people, one bay

Double booking is the problem customers notice first, and it's harder than it looks. "Check if the slot is free, then insert" has a gap between the two steps, and with two app instances behind a load balancer, an in-memory lock doesn't help.

I pushed the rule down into PostgreSQL with an **exclusion constraint**:

```
constraint ex_appointment_bay_overlap exclude using gist (  
    bay_id with =,  
    tstzrange(slot_start, slot_end, '[') with &&  
) where (status <> 'CANCELLED')
```

In plain English: *no two live bookings on the same bay may have overlapping time ranges*. The database enforces it for every writer, whether that's a buggy code path, a second instance or a hand-written SQL script. The service tries the likely-free bays in turn; losing a race is just an error code that means "try the next bay", and when none are left the customer gets a clean `409 Slot unavailable`.

I wrote an integration test using Testcontainers: 20 threads, one slot, 2 bike stands, and exactly 2 bookings expected. It passed. I was happy.

Bug 1: the test that couldn't fail

Then I wrote a k6 load test: 50 virtual users booking the same bike slot at a branch with 4 bike stands. The result came back **50 bookings out of 50**.

That's impossible with 4 stands, which made it the most useful number of the day. The bug was in the test: k6 runs your initialisation code once *per virtual user*, and I'd picked the date with `Math.random()` there. Every user was booking a different day, so nothing competed for anything.

Lesson: a test that can't fail proves nothing. Check that the number you got is actually possible.

Bug 2: the deadlock that only appears at scale

With the date fixed so all 50 users really competed, the result was worse: **1 booking, 49 HTTP 500 errors, and availability reads slowed to 23 seconds**.

The Postgres log explained it:

```
ERROR: deadlock detected  
DETAIL: Process 107 waits for ShareLock on transaction 916; blocked by process 115.  
        Process 115 waits for ShareLock on transaction 918; blocked by process 107.  
CONTEXT: while checking exclusion constraint on tuple (2,6) in relation "appointment"
```

When two transactions insert conflicting rows at the same instant, each one checks the constraint, sees the other's uncommitted row, and waits for it to finish. Both wait, forever, until Postgres picks a victim after its one-second deadlock

timeout. Meanwhile every waiting request holds a database connection, so the pool runs dry and even simple reads queue behind them.

My 20-thread test had missed it because its default 10-connection pool quietly limited how many inserts ran at once. So before fixing anything, I made the test fail: 50 threads, a 60-connection pool, and the same 4-bay scenario k6 used. It failed with `deadlock detected`, which proved it was now a real regression test.

The fix is one line before the insert:

```
jdbc.query("select pg_advisory_xact_lock(?, ?)", rs -> null, BAY_LOCK_NAMESPACE, bay.getId().intValue());
```

A transaction-scoped advisory lock *per bay* makes inserts for the same bay queue briefly instead of deadlocking. Each one after the winner then fails fast with a clean constraint violation. Each transaction holds at most one such lock, so no cycle can form, and different bays and branches don't wait for each other at all. The constraint is still what guarantees correctness; the lock only removes the deadlock.

After the fix:

	Before	After
Bookings for 4 bays under a 50-way race	1	4
Clean <code>409 Slot unavailable</code>	0	46
Server errors	49	0
Availability p95 at 200 req/s	23 s	2.6 ms

Keeping reads fast

Customers browse availability far more often than they book, so the appointment service caches each branch's booked intervals per day in Redis for 60 seconds and computes availability in memory. Caching raw intervals rather than finished answers means one cache key per branch-day, and one delete after any booking.

The cache is only an optimisation. If it's slightly stale, a customer might see a slot that's just been taken; the booking then fails cleanly with a 409 and the grid refreshes. If Redis is down, reads fall back to PostgreSQL. The database always makes the final decision.

The front end

The UI is React 19 with TypeScript, with a screen for each role: **Book service** for customers, **Workshop** for advisors (a live board from *Checked in* to *Ready for pickup*), and **Inventory** for the parts desk. A bell icon shows every SMS and email the system has sent, which is a nice way to see Kafka events arriving.

Even printing an invoice had a lesson in it. My first attempt printed the invoice dialog by hiding everything else with print CSS, and it produced a blank page, because the dialog lived inside the part being hidden. The robust answer was a dedicated printable invoice page, which I verified by printing to PDF in headless Chrome rather than trusting a screenshot.

Working with an AI assistant

I used an AI coding assistant for much of the design and implementation, and I kept a [review log](#) of its suggestions and what I did with each. Some were good and went in as written. Several were rejected: a Redis lock for booking (it can be lost on failover), publishing to Kafka straight after saving (the dual-write problem), and `ddl-auto=update` instead of reviewed migrations. The most valuable entries are the ones where the suggestion *looked* right and only measurement showed otherwise, like the deadlock.

The skill that mattered most wasn't writing code faster. It was asking "how would I know if this is wrong?" and then building the test that answers it.

Testing

- **59 backend tests**: unit tests for the domain rules and pricing, MockMvc tests for the HTTP contract, and Testcontainers races against a real PostgreSQL database
- **29 front-end tests** with Vitest and React Testing Library
- a 13-step **end-to-end script** that books, checks in, runs the parts saga both ways and invoices
- a **k6** race-and-throughput test with pass/fail thresholds

Try it

Everything runs locally with Docker; you don't need Java, Maven or Node installed:

```
make up      # then open http://localhost:3000
```

What I'd do next

- deploy to AWS with Kubernetes (EKS, RDS, MSK, ElastiCache)
- dashboards for booking conflicts, outbox lag and consumer lag
- sign-in with customer and advisor roles
- no-show handling that frees a bay automatically

Tech stack: Java 21, Spring Boot 4, Spring Cloud Gateway, Hibernate 7, Flyway, PostgreSQL 17, Apache Kafka 3.9, Redis 7, React 19, TypeScript, Vite, Vitest, JUnit 5, Mockito, Testcontainers, k6, Docker Compose, GitHub Actions.